// Day 5 of 10

Day 5

10 Git Commands

Developers make these every day.
Don't be one of them.

→ Swipe through all 10

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #01

git stash

Save work without committing

```
// Bad
// Making unrelated fix
// Don't want to commit yet — stuck!

// Good
git stash           # save work
git stash pop       # restore it
git stash list      # see all stashes
```

💡 Stash is your temporary save slot

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #02

git log --oneline

Clean commit history view

```
// Bad
git log
// Huge output, hard to read

// Good
git log --oneline --graph
// a3f8c2d Fix login bug
// b2e1f0a Add user API
```

💡 Add --graph for branch visualization

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #03

git rebase -i

Clean up messy commits before PR

```
// Bad
// fix typo
// fix typo again
// fix typo AGAIN

// Good
git rebase -i HEAD~3
// Squash 3 commits into 1
// Clean history
```

💡 Never rebase on shared branches

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #04

git bisect

Find which commit broke things

```
// Bad
// Something broke
// Check 50 commits manually?

// Good
git bisect start
git bisect bad            # now = broken
git bisect good v1.0      # v1.0 = good
// Git finds the bad commit!
```

💡 Saves hours of debugging

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #05

git cherry-pick

Apply a specific commit anywhere

```
// Bad
// Need one fix from another branch
// Merge whole branch? No!

// Good
git cherry-pick a3f8c2d
// Applies just that one commit
// to your current branch
```

💡 Perfect for hotfixes across branches

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #06

git reflog

Recover lost work

```
// Bad
git reset --hard HEAD~3
// Lost 3 commits!

// Good
git reflog
// Shows all recent HEAD movements
git checkout a3f8c2d
// Your work is back!
```

💡 Reflog saves you from mistakes

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #07

.gitignore Properly

Never commit secrets or build files

```
// Bad
// Committed node_modules/
// Committed .env file
// Committed .class files

// Good
node_modules/
.env
*.class
target/
.DS_Store
```

💡 Use gitignore.io by tech stack

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #08

git diff --staged

Review before you commit

```
// Bad
git add .
git commit -m "changes"
// What did I actually commit?

// Good
git add .
git diff --staged
// See exactly what's staged
// THEN commit with confidence
```

💡 Always review staged changes first

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #09

Meaningful Commit Messages

Your future self will thank you

```
// Bad
git commit -m "fix"
git commit -m "changes"
git commit -m "asdklj"

// Good
git commit -m "fix: handle null user in login"
git commit -m "feat: add pagination"
git commit -m "docs: update README"
```

💡 Use: fix/feat/docs/refactor prefix

Shubham Bhati
linkedin.com/in/bhatishubham

Mistake #10

git tag for Releases

Mark important versions clearly

```
// Bad
// Which commit is v1.0?
// No idea without tags

// Good
git tag -a v1.0.0 -m "First release"
git push origin v1.0.0
// Everyone knows what v1.0 is!
```

💡 Semantic versioning: vMAJOR.MINOR.PATCH

Shubham Bhati
linkedin.com/in/bhatishubham

```
done = true
```

Follow for more tips!

Like · Comment · Repost

Shubham Bhati
linkedin.com/in/bhatishubham